

무작위성 (Randomness)

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. 무작위성 (Randomness)
2. 조건문 (Conditional statements)
3. 반복 (Iteration)
4. 시뮬레이션 (Simulation)
5. 몬티 홀 문제 (Monty hall problem)
6. 확률 구하기 (Finding probabilities)

무작위성

무작위성 (1)

■ 무작위성

- 무작위로 처치(treatment) 그룹과 대조(control) 그룹에 배정한 뒤, 두 그룹 결과의 차이가 단순히 무작위 배정 때문에 생긴 것인지, 아니면 실제 treatment 효과 때문인지를 판단할 수 있어야 함
- 무작위성 선택 in 파이썬

- np.random.choice

```
two_groups = make_array('treatment', 'control')  
np.random.choice(two_groups)
```

'treatment'

'treatment'와 'control' 각 50% 확률로 선택

```
np.random.choice(two_groups, 10)
```

```
array(['control', 'control', 'treatment', 'treatment', 'control',  
      'control', 'control', 'control', 'control', 'control'], dtype='<U9')
```

무작위 선택 10번 시행

- 무작위성 질문 예시들
 - 어떤 사람이 treatment 그룹에 배정되었는가?
 - 도박사가 돈을 딸 것인가?
 - 여론조사가 정확한 예측을 했는가?

무작위성 (2)

■ 불과 비교 (Boolean and Comparison)

- 비교의 결과는 Boolean 형태로 반환 (in 파이썬)

```
3 > 1 + 1
```

- 문자열 비교

True

- 비교 가능하며, 비교 순서는 알파벳 순서를 따름

```
'Dog' > 'Catastrophe' > 'Cat'
```

True

```
np.random.choice(two_groups) == 'treatment'
```

True

■ 배열과 값 비교하기

- 배열 안의 요소와의 비교 가능

```
tosses = make_array('Tails', 'Heads', 'Tails', 'Heads', 'Heads')  
tosses == 'Heads'
```

array([False, True, False, True, True])

```
np.count_nonzero(tosses == 'Heads')
```

3

조건문

조건문

■ 조건문 (Conditional statements)

- 행동(actions)과 결과(results)는 특정 조건(condition)이 만족되는지 여부에 따라 달라짐
 - (예시1) 무작위 대조 실험(randomized controlled trial)에서 사람들은 처리(treatment) 그룹에 배정되었을 때만 처리 진행
 - (예시2) 도박사는 자신의 베팅에서 이겼을 때 돈을 벌

■ (예제) 주사위 도박

- If the die shows 1 spot or 2 spots, I lose a dollar.
- If the die shows 3 spots or 4 spots, I neither lose money nor gain money.
- If the die shows 5 spots or 6 spots, I gain a dollar.

```
def one_bet(x):  
    """Returns my net gain if the die shows x spots"""  
    if x <= 2:  
        return -1  
    elif x <= 4:  
        return 0  
    elif x <= 6:  
        return 1
```

```
one_bet(17)
```

➔ 값은 범위를 넘어섬

```
one_bet(np.random.choice(np.arange(1, 7)))
```

-1

반복

반복 (1)

■ 반복 (Iteration)

- 무작위성을 다룰 때, 어떤 과정을 여러 번 반복해야 하는 경우 존재
- for문 사용

```
def bet_on_one_roll():  
    """Returns my net gain on one bet"""  
    x = np.random.choice(np.arange(1, 7)) #  
    if x <= 2:  
        return -1  
    elif x <= 4:  
        return 0  
    elif x <= 6:  
        return 1
```

```
for i in np.arange(5):  
    print(bet_on_one_roll())
```

```
1  
-1  
-1  
1  
1
```

■ 배열 확장하기 (Augmenting arrays)

- 반복 결과를 저장할 때 배열을 사용

```
pets = make_array('Cat', 'Dog')  
np.append(pets, 'Another Pet')
```

→ 원래 배열은
변하지 않음 !!

```
array(['Cat', 'Dog', 'Another Pet'], dtype='<U11')
```

```
pets
```

```
array(['Cat', 'Dog'], dtype='<U3')
```

```
pets = np.append(pets, 'Another Pet')  
pets
```

```
array(['Cat', 'Dog', 'Another Pet'], dtype='<U11')
```

반복 (2)

■ (예제) 5번 주사위 베팅하기

```
outcomes = make_array()

for i in np.arange(5):
    outcome_of_bet = bet_on_one_roll()
    outcomes = np.append(outcomes, outcome_of_bet)

outcomes
```

array([-1., -1., 1., 1., -1.])

```
np.count_nonzero(outcomes)
```

3

```
outcomes = make_array()

i = np.arange(5).item(0)
outcome_of_bet = bet_on_one_roll()
outcomes = np.append(outcomes, outcome_of_bet)

i = np.arange(5).item(1)
outcome_of_bet = bet_on_one_roll()
outcomes = np.append(outcomes, outcome_of_bet)

i = np.arange(5).item(2)
outcome_of_bet = bet_on_one_roll()
outcomes = np.append(outcomes, outcome_of_bet)

i = np.arange(5).item(3)
outcome_of_bet = bet_on_one_roll()
outcomes = np.append(outcomes, outcome_of_bet)

i = np.arange(5).item(4)
outcome_of_bet = bet_on_one_roll()
outcomes = np.append(outcomes, outcome_of_bet)

outcomes
```

array([1., 0., 0., -1., 1.])

반복 (3)

■ (예제) 300번 주사위 베팅하기

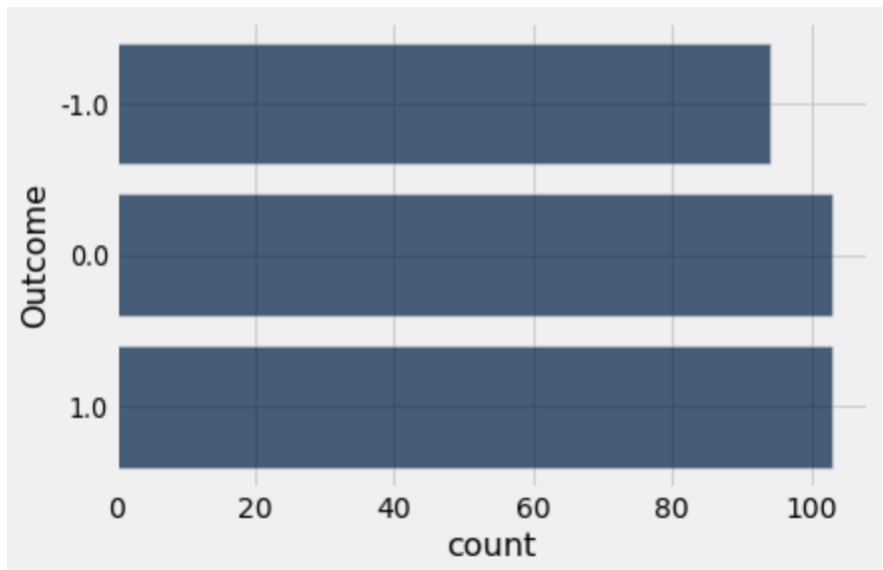
```
outcomes = make_array()

for i in np.arange(300):
    outcome_of_bet = bet_on_one_roll()
    outcomes = np.append(outcomes, outcome_of_bet)
```

```
len(outcomes)
```

300

```
outcome_table = Table().with_column('Outcome', outcomes)
outcome_table.group('Outcome').barh(0)
```



시뮬레이션

시뮬레이션 (1)

- 시뮬레이션 (Simulation)

- 컴퓨터를 사용해 실제 물리적 실험을 흉내내는 과정
- 시뮬레이션 과정 (Process) 4단계
 - ① 무엇을 시뮬레이션 할 것인가? (What to simulate?)
 - 시뮬레이션하고 싶은 양을 결정
 - (예) 동전 던지기 결과
 - ② 하나의 값 시뮬레이션 하기 (Simulating one value)
 - 1단계에서 정한 양 하나를 어떻게 시뮬레이션 할지 결정
 - ③ 반복 횟수 결정 (Number of repetitions)
 - 양을 몇 번 시뮬레이션 할 지 결정
 - (예) 동전 1000번 던지기
 - ④ 여러 값 시뮬레이션 하기 (Simulating multiple values)
 - 시뮬레이션 시도 및 저장

시뮬레이션 (2)

- (예제) 동전 100번 던졌을 때 앞면 개수

- (1단계) 동전 100번 던졌을 때 앞면 개수

- (2단계) 하나의 값 시뮬레이션

```
coin = make_array('Heads', 'Tails')
```

```
ten_tosses = np.random.choice(coin, 10)
ten_tosses
```

```
array(['Heads', 'Tails', 'Heads', 'Tails', 'Tails', 'Heads', 'Tails',
       'Tails', 'Tails', 'Heads'], dtype='<U5')
```

```
np.count_nonzero(ten_tosses == 'Heads')
```

4

```
outcomes = np.random.choice(coin, 100)
num_heads = np.count_nonzero(outcomes == 'Heads')
num_heads
```

46

```
def one_simulated_value():
    outcomes = np.random.choice(coin, 100)
    return np.count_nonzero(outcomes == 'Heads')
```

- (3단계) 반복 횟수 결정: 100번 or 20,000번

- (4단계) 여러 값 시뮬레이션 및 저장

```
num_repetitions = 20000 # number of repet
```

```
heads = make_array() # empty collection arr
```

```
for i in np.arange(num_repetitions): # re
    new_value = one_simulated_value() # si
    heads = np.append(heads, new_value) # a
```

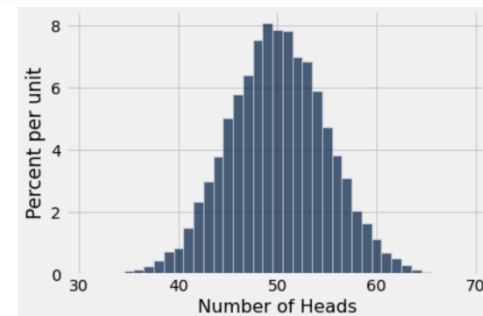
```
# That's it! The simulation is done.
```

```
simulation_results = Table().with_columns(
    'Repetition', np.arange(1, num_repetitions + 1), ... (19997 rows omitted)
    'Number of Heads', heads
)
```

```
simulation_results.show(3)
```

Repetition	Number of Heads
1	44
2	54
3	44

```
simulation_results.hist('Number of Heads', bins = np.arange(30.5, 69.6, 1))
```



시뮬레이션 (3)

▪ (예제) 모노폴리 이동 수

- 한번 이동 (move) = 두 번 주사위 굴린 눈의 합 or 2개의 주사위를 동시에 굴린 눈의 합
- (1단계) 두 번 주사위 굴린 눈의 합
- (2단계) 한번 이동할 때의 시뮬레이션 수
- (3단계) 10,000번
- (4단계) 시뮬레이션 시도 및 결과 저장

```
die = np.arange(1, 7)
sum(np.random.choice(die, 2))
```

7

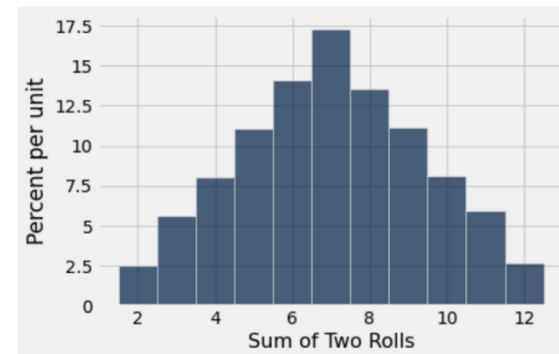
```
def one_simulated_move():
    return sum(np.random.choice(die, 2))
```

```
num_repetitions = 10000

moves = make_array()
for i in np.arange(num_repetitions):
    new_move = one_simulated_move()
    moves = np.append(moves, new_move)
```

```
results = Table().with_columns(
    'Repetition', np.arange(1, num_repetitions + 1),
    'Sum of Two Rolls', moves
)

results.hist('Sum of Two Rolls', bins = np.arange(1.5, 12.6, 1))
```



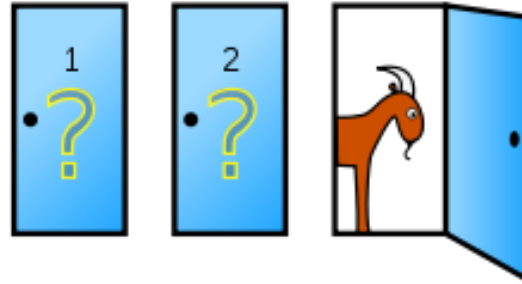
몬티 홀 문제

몬티 홀 문제 (1)

- 몬티 홀 문제 (Monty hall problem)

- 상황

- 한 문 뒤에는 멋진 자동차
 - 나머지 두 문 뒤에는 염소



- 게임 룰

- 자동차를 찾는 문제
 - (참가자) 문 하나를 선택 ➔ 하지만, 그 문은 아직 열지 않음
 - 남은 두 문 중 적어도 하나 뒤에는 염소가 있음
 - (Monty) 그 염소가 있는 문 하나를 열어 염소를 보여줌
 - 남은 두 문 중 하나 뒤에는 자동차, 하나 뒤에는 염소
 - (참가자) 둘 중 어떤 문을 열지 다시 선택
 - 처음 선택을 유지해야 할까?
 - 다른 문으로 바뀌어야 할까?

몬티 홀 문제 (2)

■ 몬티 홀 문제 (Monty hall problem) (2)

■ 정답

- 처음 선택한 문이 자동차일 확률이 1/3
- 남아 있는 문 뒤에 자동차가 있을 확률은 2/3 → 참가자는 문을 바꾸는 것이 유리

■ 시뮬레이션

- (1단계) 무엇을 시뮬레이션 할 것인가? → 하나가 아니라 세개의 양(quantity)
 - 참가자가 처음 고른 문의 뒤의 것
 - Monty가 연 문 뒤의 것
 - 마지막 남은 문의 것
- (2단계) 한번의 게임 시뮬레이션하기

```
goats = make_array('first goat', 'second goat')
```

```
def other_goat(x):  
    if x == 'first goat':  
        return 'second goat'  
    elif x == 'second goat':  
        return 'first goat'
```

```
hidden_behind_doors = np.append(goats, 'car')  
hidden_behind_doors
```

```
def monty_hall_game():  
    """Return  
    [contestant's guess, what Monty reveals, what remains behind the  
  
    contestant_guess = np.random.choice(hidden_behind_doors)  
  
    if contestant_guess == 'first goat':  
        return [contestant_guess, 'second goat', 'car']  
  
    if contestant_guess == 'second goat':  
        return [contestant_guess, 'first goat', 'car']  
  
    if contestant_guess == 'car':  
        revealed = np.random.choice(goats)  
        return [contestant_guess, revealed, other_goat(revealed)]
```

```
monty_hall_game()
```

```
['car', 'first goat', 'second goat']
```

몬티 홀 문제 (3)

■ 몬티 홀 문제 (Monty hall problem) (3)

■ 시뮬레이션

- (3단계) 10,000번 반복
- (4단계) 시뮬레이션 하기 및 결과 저장

```
# empty collection table
games = Table(['Guess', 'Revealed', 'Remaining'])

# Play the game 10000 times and
# record the results in the table g

games.show(3)

for i in np.arange(10000):
    games.append(monty_hall_game())
```

Guess	Revealed	Remaining
first goat	second goat	car
first goat	second goat	car
car	first goat	second goat
... (9997 rows omitted)		

```
original_choice = games.group('Guess')
original_choice
```

Guess	count
car	3319
first goat	3311
second goat	3370

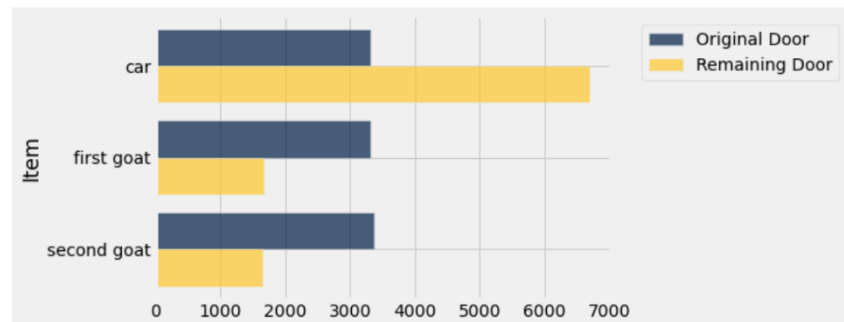
```
remaining_door = games.group('Remaining')
remaining_door
```

Remaining	count
car	6681
first goat	1676
second goat	1643

```
joined = original_choice.join('Guess', remaining_door, 'Remaining')
combined = joined.relabeled(0, 'Item').relabeled(1, 'Original Door').relabeled(2, 'Remaining Door')
combined
```

Item	Original Door	Remaining Door
car	3319	6681
first goat	3311	1676
second goat	3370	1643

```
combined.barh(0)
```



확률 구하기

확률 구하기 (1)

- 확률의 정의
 - 상대 도수(relative frequency) → 수업에서는 상대도수를 확률로 사용
 - 장기적인 상대도수(long run relative frequency)
 - 자신의 불확실성 정도를 나타내는 주관적 척도(subjective measure)

- 사건이 일어나지 않을 때

$$P(\text{an event doesn't happen}) = 1 - P(\text{the event happens})$$

- 모든 결과가 동일한 확률 일 때

$$P(\text{an event happens}) = \frac{\#\{\text{outcomes that make the event happen}\}}{\#\{\text{all outcomes}\}}$$

(Ex) $\frac{\text{number of even faces}}{\text{number of all faces}} = \frac{\#\{2, 4, 6\}}{\#\{1, 2, 3, 4, 5, 6\}} = \frac{3}{6}$ $P(\text{die shows a multiple of 3}) = \frac{\#\{3, 6\}}{\#\{1, 2, 3, 4, 5, 6\}} = \frac{2}{6}$

확률 구하기 (2)

- 두 사건이 모두 일어날 때
 - 곱셈 법칙 (multiplication rule)

$$P(\text{two events both happen}) = P(\text{one event happens}) \\ \times P(\text{the other event happens, given that the first one happened})$$

(Ex)

$$P(\text{green first, then red}) = \frac{\#\{\text{GR}\}}{\#\{\text{RB, BR, RG, GR, BG, GB}\}} = \frac{1}{6}$$

$$P(\text{green first, then red}) = \frac{1}{3} \times \frac{1}{2} = \frac{1}{6}$$

- 사건이 두 가지 방식으로 일어날 수 있을 때
 - 덧셈 법칙 (addition rule)

$$P(\text{an event happens}) = P(\text{first way it can happen}) + P(\text{second way it can happen})$$

(Ex)

$$P(\text{one green and one red}) = P(\text{GR}) + P(\text{RG}) = \frac{1}{6} + \frac{1}{6} = \frac{2}{6}$$

확률 구하기 (3)

■ 적어도 한번의 성공

- 모집단(population)에서 랜덤 표본(random sample)을 다룸
- (질문) 특정 개인이 표본에 포함될 확률은?
 - 이 확률을 계산할 때 → 그 개인을 'success'라고 부름
 - 즉, 표본 안에 'success'가 포함될 확률을 구하는 것

- (예제) 동전 두 번 던지기 – 적어도 한 번 앞면 (head)이 나올 확률

$$P(\text{at least one head in two tosses}) = 1 - P(\text{both tails}) = 1 - \frac{1}{4} = \frac{3}{4}$$

$$P(\text{both tails}) = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2} = \left(\frac{1}{2}\right)^2$$

- (예제) 17번 던져 적어도 한번 앞면 (Head)

$$P(\text{at least one head in 17 tosses}) = 1 - P(\text{all 17 are tails}) = 1 - \left(\frac{1}{2}\right)^{17}$$

$$P(\text{at least one 6 in 17 rolls}) = 1 - \left(\frac{5}{6}\right)^{17}$$

```
rolls = np.arange(1, 51, 1)
results = Table().with_columns(
    'Rolls', rolls,
    'Chance of at least one 6', 1 - (5/6)**rolls
)
results
```

Q & A
